Project 05 — Workplace Safety Tracking on ONNX Runtime

IoT / edge · multi-object detection and tracking · real-time zone intrusion

Contents

Project 05 — Workplace Safety Tracking (ONNX Runtime)

> Read `00-shared-requirements.md` first. Everything in it applies here, with the device-class substitutions in §4.

---

0. Numbered build plan

Paste this section into Claude Code's plan mode to scope the work. Each step is independently reviewable; steps 1–4 must land before the demo is filmable.

1. **Repository skeleton.** Monorepo with `frontend/`, `backend/`, `model/`, `deploy/`, `observability/`. Strict TypeScript, typed Python, lint + types + tests in CI from the first commit.
2. **ONNX Runtime serving layer.** Stand up the engine with the compression spec in §3–§4 below. Record weights and latency before and after, and state what the freed resource buys in the units the client pays for.
3. **Backend API.** The component chain in §5 as real services: authentication, authorization, adversarial-input detection, context assembly, inference, guardrails, structured logging, hash-chained audit log.
4. **Product tab.** The working demo, usable by someone who has never seen it, plus the configuration surface. Configuration changes must visibly change behaviour and be inspectable before they apply.
5. **Architecture tab.** The 3D expandable-pipeline graph, built to the spec in `00-shared-requirements.md` §2. Use the node inventory in §7 of this document as the content. Click-to-isolate, draggable nodes, canvas-drawn icon glyphs, constant-size labels, HTML detail panel with all four rationale fields plus a "Say this" line.

6. **Guided tour.** An ordered walk covering every node that carries the commercial argument, including the honest stop about what is *not* enabled. Arrow keys, autoplay, and it must run with the backend stopped.
7. **Monitoring tab.** Quality, Performance, Security, Audit, Improvement. Live data when available, seeded demo data otherwise, honest `source` badge — except Improvement, which is never seeded.
8. **Feedback capture.** Rating, side-by-side comparison and correction in the Product tab, per §8. Redact on write, stamp the model/config version, store the triple denormalised, mark what an export consumed.
9. **Export and training path.** JSON Lines in the trainer's own format, resumable. Training is operator-run, never automatic. Write a training manifest next to every artifact. Evaluate **LoRA adapters** per §8a — that section says where they belong in *this* project and where they do not.
10. **Release gate.** Both axes per §9 — quality and performance, each able to fail the build on its own. Wire it into CI.
11. **Deployment.** Containers, infrastructure as code, staging production with a rollback, and autoscaling on the signal that actually saturates.
12. **README.** A stranger can reach a working demo. Ends with the cost/performance/accuracy tradeoff table, one row per real decision, and a portability section.

Acceptance is §13 of this document plus the shared deliverables checklist.

---

1. Brief

Build an **edge safety monitor** for industrial floors and warehouses. Cameras watch a working area; the system detects people and vehicles, tracks them frame to frame, and raises an alert when someone enters a hazard zone or a forklift and a pedestrian converge.

The defining constraint:

> **Video never leaves the site.** Inference runs on a device on the local network. The only things that travel are anonymous events — "person entered zone 3 at 14:22, dwell 4 s" — never frames, never identities.

That is both the privacy design and the economic one. Streaming twelve cameras to the cloud costs bandwidth continuously and creates a video archive somebody has to govern; processing locally and emitting events costs neither.

Design it as a safety system, not a surveillance system. It counts and tracks anonymous objects — no face recognition, no identification, no per-worker productivity metrics. That boundary is what makes it deployable in workplaces with works councils or union agreements, and it should be enforced in the architecture, not just promised.

Target user: manufacturing, logistics, construction, ports — sites with vehicle and pedestrian mixing, PPE requirements and a genuine incident rate.

---

2. Why ONNX Runtime for this workload

- **One graph, many accelerators.** Edge deployments are heterogeneous by nature: some sites have a Jetson, some an Intel NUC, some an ARM box already installed. ONNX Runtime's **execution providers** — TensorRT, OpenVINO, CUDA, DirectML, CoreML, plain CPU — run the same `.onnx` file on all of them, choosing the fastest available at load time. Maintaining one model artifact across a mixed fleet is a large operational saving and the honest headline reason to choose it.
- **Mature INT8 quantization tooling** with both static (calibrated) and dynamic paths, plus per-channel weight quantization, which detection models need.
- **Graph optimisations** — constant folding, node fusion, layout transforms — applied at session creation with no code change.
- **It is a stable, boring, well-supported runtime.** For a device bolted to a wall in a warehouse for three years, that is a feature.

---

3. No KV cache here — the substitution

Per shared requirements §4, state this on the architecture node:

> A detector runs **one forward pass per frame**. There is no autoregressive generation, no KV state, no KV cache, and no continuous batching in the LLM sense.

What replaces it, and what the demo must show instead:

Technique	What it buys
INT8 static quantization (calibrated on site footage)	~4× smaller, 2–4× faster; integer units are what edge NPUs are built for
Execution provider selection	TensorRT on Jetson, OpenVINO on Intel — same file
Graph fusion	Conv+BN+activation folded; fewer memory round trips
Input resolution	The largest latency lever in vision — and a direct recall trade on small/distant objects
Frame batching across cameras	Genuinely applicable here: batch N camera frames into one forward pass. This *is* the batching story, and it is real
Frame skipping + tracker interpolation	Detect at 10 FPS, track at 30 — the biggest single cost reduction available

That last one deserves emphasis in the client-facing copy: **the tracker is essentially free compared to the detector**, so running detection on every third frame and letting the tracker carry the intermediate frames cuts compute by roughly two thirds with little accuracy loss on slow-moving subjects. It is the highest leverage decision in the project and exactly the kind of engineering judgement a client is paying for.

---

4. Models and compression

Stage	Model	Notes
Detection	YOLOv8n / YOLO11n exported to ONNX	Classes: person, forklift, vehicle, and PPE items if required
Tracking	ByteTrack or OC-SORT	CPU, no learned model, cheap and robust
Optional	Pose model for fall detection	Only if the site's incident profile justifies it

Compression spec - INT8 static quantization calibrated on **footage from the actual site** — lighting, camera angle, motion blur and occlusion patterns differ enormously between a warehouse and a dockside, and calibration data from the wrong environment leaves accuracy on the table. - Record per execution provider: model size, FPS, mAP delta, power draw. - Ship an FP16 variant for comparison; on some accelerators FP16 is as fast as INT8 with less accuracy loss, and you cannot know without measuring.

Quality gate - mAP@50 and mAP@50-95 on a held-out set from the same site, gated as a delta against FP32. - **Recall on small and distant objects specifically.** This is where INT8 damage concentrates, and in a safety system a missed distant pedestrian is the failure that matters. Aggregate mAP hides it. - Tracking metrics: MOTA, IDF1, ID switches. A detector that is accurate per frame but swaps identities constantly produces useless dwell times and false alerts. - Sustained FPS on target hardware, measured over a long run, not a burst.

---

5. Architecture

```
Camera (RTSP) ⟶ Edge device on the local network
```

```
→ Stream ingest, decode, frame buffer
→ Device authentication (mTLS to the local hub)
→ Authorization (which cameras, which zones)
→ Frame sampling / skip policy
→ ONNX Runtime detection
     · INT8 quantized .onnx
     · execution provider: TensorRT / OpenVINO / CPU
     · multi-camera frame batching
→ Multi-object tracker (ByteTrack)
→ Zone logic + rules engine        ← dwell, intrusion, proximity, PPE
→ Confidence + persistence gate    ← N consecutive frames before alerting
→ Anonymous event emission         ← no frames, no identities
→ Stateless structured logs
→ Append-only audit log, hash-chained
→ Local metrics ⋯ sync ⋯ Prometheus → Grafana
→ Model registry → canary device → staged fleet rollout   ← replaces Kubernetes

                       → Fleet rollout (canary cameras, signed artifacts)

══ IMPROVEMENT LOOP (the RLHF analogue) ═══════════════════════════

Missed detection  → reviewer draws a box  → that box is a label
False detection   → reviewer rejects a box
ID switch         → reviewer marks the frame (TRACKER signal,
kept separate from detector feedback)
→ store the FRAME with the correction, at model resolution
→ Retraining set + class-level scoring ⟶ Grafana
→ Retrain (operator-run, never automatic)
→ Promotion gate: per-class mAP suite + sustained-FPS harness

==> back to the ONNX detector
```

Build deliberately:

- **The persistence gate is what makes it usable.** Alerting on a single frame's detection produces constant false alarms, and a safety system that cries wolf gets switched off within a week — which is the real failure mode of this product category. Require N consecutive frames, and make N configurable per rule.
- **Zones are drawn on the camera image, not typed as coordinates.** A site manager must be able to configure this without an engineer. Put the zone editor in the Product tab.
- **Events are anonymous by construction.** Track IDs are ephemeral and reset per session. They exist to link a person's frames within one event, not to follow anyone across time — and the architecture should make persistent tracking impossible rather than merely disabled.
- **Store nothing by default.** Optionally retain a short pre/post clip on an alert for incident review, with retention and access control stated. That is the one place video is kept, and it should be a deliberate, visible exception.

---

6. Frontend

Product tab — live camera view with detection boxes, track IDs and trails, zone overlays, and an event feed. **A zone editor** — draw a polygon on the image, set the rule (no-entry, dwell limit, proximity threshold), set severity. An alert timeline.

Configuration: cameras, zones, rules, classes tracked, confidence and persistence thresholds, detection frame rate.

Architecture tab — 3D graph. Logos: ONNX, NVIDIA Jetson, Intel OpenVINO, Prometheus, Grafana. Strongest copy on **INT8 quantization**, **execution provider selection**, **frame skipping + tracker interpolation**, **the persistence gate**, and the **no-KV-cache** node explaining the substitution.

Monitoring tab — see below.

The Architecture tab must show the improvement loop. It is a stage card like any other, and it is the only one whose edges flow *back* upstream — that is the visual point. Give it the green treatment and its own tour stops; see §7 for the node inventory and `00-shared-requirements.md` §2 for the graph mechanics (expandable pipeline, click-to-isolate, draggable nodes, canvas-drawn icon glyphs, constant-size labels, HTML detail panel).

The Monitoring tab has five sections, not four — Quality, Performance, Security, Audit and **Improvement**. The last one is never seeded with demo data.

---

7. Architecture graph: node inventory

These are the stage cards for this project's Architecture tab, in pipeline order. Every one is a node; every child is a node inside its parent's card. Build them against the `ArchNode` shape in `00-shared-requirements.md` §8a, with all four rationale fields and a "Say this" line on each.

#	Stage card	Colour role	Expands into
0	**Camera**	neutral	— (the stream, and whatever it is pointed at)
1	**Viewer Console**	teal	Live overlay, zone editor, class selection, incident review
2	**Ingest & Security**	amber	RTSP/WebRTC ingest, TLS, authentication, authorization, stream-source validation, rate limiting
3	**Frame Pipeline**	violet	Decode, resize, letterbox, normalisation, frame skipping policy
4	**ONNX Runtime Detector**	orange	**YOLO-class detector (INT8 static quantization)**, graph fusion, execution provider selection, IO binding, dynamic batching, **"no KV cache — and why"**
5	**Tracker**	violet	Detection-to-track association, Kalman prediction, ID management, track lifecycle
6	**Rules & Zones**	violet	Zone occupancy, dwell time, line crossing, alert policy, confidence gating
7	**Database**	blue	PostgreSQL, track and event store, model registry
8	**Monitoring & Audit**	magenta	Prometheus, Grafana, hash-chained audit log, retention policy, structured logging

#	Stage card	Colour role	Expands into
9	**Improvement Loop**	green	Missed/false detection reporting from review, corrected-frame queue, class-level scoring, retraining set, promotion gate mAP suite, FPS harness
10	**Fleet**	grey	Edge device fleet, canary cameras, signed model artifacts, staged rollout, rollback

The card carrying the argument is **ONNX Runtime Detector**, and the honest framing is throughput per pound rather than raw accuracy: INT8 plus graph fusion plus the right execution provider is the difference between one box handling four cameras and one box handling sixteen. That ratio is the number the buyer is actually purchasing.

ID switches, not mAP, are what users notice. A tracker that swaps two people's identities as they cross produces a report nobody trusts, however good the detector's mAP is. Give the tracker its own strong detail-panel copy.

---

8. The improvement loop for this project

`00-shared-requirements.md` §5a applies in full. What is specific here:

The feedback here is **spatial**, which changes how it must be captured:

- **Missed detection** — the reviewer draws a box on a frame where nothing was detected. That box is a training label.
- **False detection** — the reviewer rejects a box. Equally valuable, and much more common in practice.
- **ID switch** — the reviewer marks the frame where two tracks swapped. This is the tracker's signal, not the detector's, and conflating them means tuning the wrong component.

Rules for this domain:

- **Store the frame with the correction**, at the resolution the model saw. A label without its pixels is not a training example, and re-decoding the source video later is unreliable once retention has run.
- **Separate detector feedback from tracker feedback.** They train different things; a queue that mixes them teaches neither.
- **Sample the boring frames too.** A dataset built only from frames someone bothered to correct is biased toward hard cases, and a model trained on it becomes over-eager on easy ones.
- **Retention and faces.** If the frames contain identifiable people, the correction queue is personal data. Blur non-target regions on write, or set a short retention and say what it is.

8a. LoRA adapters: where they earn their place here

Test this, but scope it honestly.

ONNX Runtime executes a frozen, fused, quantized graph. There is no adapter swapping at inference time, and no per-request adapter routing. LoRA belongs to the training half of this project:

- Adapt the detector backbone with LoRA in PyTorch, **merge, then export to ONNX and quantize**. What runs on the edge box is a single fused graph.
- Do not draw an adapter node in the serving path of the architecture graph. Draw it in the improvement loop, where it belongs.

Where it genuinely earns its place: per-site adaptation. Detector accuracy on a fixed camera is dominated by that camera's specifics — mounting angle, lens, lighting, what the floor looks like, what people

wear. A model that is excellent on a public benchmark is frequently mediocre on one particular loading bay, and generic retraining to fix one site risks the other forty.

LoRA makes per-site adaptation affordable to try:

Approach	Cost to adapt one site	Risk to other sites
Retrain the whole detector	high	high — a global change for a local problem
LoRA on the backbone, merged per site	low	**contained: one site's artifact**

The fleet already ships signed per-device artifacts, so a site-specific model is a rollout target you have anyway.

Measure: per-class mAP on that site's held-out frames *and* on the general benchmark, before and after. An adaptation that gains 6 points locally and loses 4 globally is usually the wrong trade, and only measuring locally hides it completely.

---

9. The release gate for this project

`00-shared-requirements.md` §5b applies in full: two axes, either one able to block the release on its own.

Dimension	Question	Tool	Thresholds that matter here
Quality	Is it still right?	Held-out detection suite: **mAP@50 and mAP@50-95 per class**, plus tracking metrics (MOTA, IDF1, ID switches)	Gate **per class**, not on the aggregate. A quantization step that costs 1 point of mAP overall while destroying the smallest and most important class is a regression the aggregate hides
Performance	Is it still fast enough for real time?	FPS harness on the target hardware at the real input resolution, plus p95 per-frame latency and memory	**Sustained FPS at the deployed resolution is an absolute floor.** Below real time the tracker starts missing associations and ID switches climb — the quality metric degrades because the performance metric did

`lm_eval` and `GuideLLM` do not apply. Substitute the two harnesses above, benchmark at the **deployed resolution and frame rate**, and run long enough for thermal throttling to appear on edge hardware.

---

10. Monitoring and KPIs

Section	Metrics
Quality	mAP, recall on small objects, ID switches per hour, **false alert rate per camera per shift**, missed-event rate against a labelled audit sample
Performance	FPS per camera, detection latency p50/p95/p99, end-to-end alert latency, GPU/NPU utilisation, **device temperature**, dropped frames
Security	Camera stream authentication failures, unauthorised zone config changes, model integrity failures, tampering (camera moved or obscured)
Audit	Every alert, every config change, every clip retained or accessed — hash-chained
Improvement	Missed/false detections reported per class, ID-switch reports, corrected frames awaiting retraining, class-level accuracy trend

The Improvement section is never seeded with demo data. Every other section falls back to synthetic numbers when the metrics backend is absent and says so with a badge. Feedback is a claim about what real people judged; a plausible approval rate nobody gave is the one number here that cannot be corrected by waiting for real traffic. An empty loop renders as empty.

False alert rate per shift is the metric the client actually cares about, and it should be the headline. Detection mAP is an engineering number; "your team got four false alarms last week instead of forty" is the business case.

Dropped frames and device temperature are early warning signs. An edge box in a warehouse ceiling will thermally throttle in summer, and the first symptom is dropped frames, not an outage. Alert on it before it becomes a missed incident.

---

11. Security, audit and compliance

- **Workplace monitoring has specific legal obligations** in many jurisdictions — works council consultation in Germany, employee notification requirements elsewhere. The system must support a **transparency report**: what is detected, what is stored, for how long, who can see it. Build it as a product feature.
- **GDPR:** even anonymous-by-design tracking can constitute personal data processing if individuals are identifiable in context. Legitimate interest is the usual basis for safety monitoring; data minimisation is served by emitting events rather than video. Document the DPIA inputs.
- **No biometric identification.** Enforced architecturally — no face model is present in the pipeline. That is a strong statement to a works council, and it is only credible if it is structurally true.
- **Camera streams authenticated with mTLS.** An unauthenticated RTSP stream on a factory network is an obvious entry point.
- **Signed model artifacts**, verified before load.
- **Retention:** events long, clips short, frames never.

---

12. Deployment

Edge fleet, not Kubernetes — though K3s on the edge boxes is a reasonable way to get declarative deployment and rollback, and worth considering.

- **Canary device □ site □ fleet** rollout rings. A bad model reaching every camera at every site simultaneously is the failure this structure prevents.
- Signed, versioned model artifacts fetched from a registry.
- **Devices must operate through network loss.** A safety alert cannot depend on a cloud round trip. Buffer events locally and sync when the link returns; alerting is local and immediate.

- Remote health check, log retrieval and rollback without a site visit — the operational cost of physically visiting devices dominates everything else at fleet scale.

---

13. Deliverables

- Working edge pipeline: RTSP in, tracked detections and alerts out
- Zone editor usable by a non-engineer
- ONNX export and INT8 quantization pipeline with a quality gate
- Execution provider benchmark table across at least two accelerators
- All three tabs
- Hash-chained audit log
- Fleet rollout with canary and rollback
- Measured FPS, latency and false alert rate on real footage

Acceptance: 1. Sustained 30 FPS effective tracking on the target device with the stated camera count, measured over an hour, not a burst 2. Quality gate fails on a model with degraded small-object recall 3. Alert fires within 2 s of a zone intrusion 4. No frames leave the device except explicitly retained incident clips 5. Device survives simulated network loss without missing local alerts 6. Architecture tab and guided tour run with the device offline

---

14. Out of scope

Face recognition or any identification of individuals — explicitly excluded. Productivity or behaviour monitoring of named workers. Re-identification across cameras or across sessions. Cloud video archive. PTZ camera control. Training a custom detector from scratch; start from pretrained weights and fine-tune if needed.